

Module 3: Performance Engineering with Python

Author: Xiaoyuan Suo, Webster University

Target Courses: CS2 (Python), Systems, Data Science, Intro HPC

Prerequisite Modules:

- Module 1: Basics of Parallel Programming in Python
- Module 2: Parallel Image Processing with Python
- Students should have basic programming skills in Python, knowing OO, functions, data structures etc.

This module introduces students to performance engineering as an engineering discipline, emphasizing measurement, analysis, explanation, and informed improvement rather than blind optimization. Using Python-based experiments, students learn why parallel programs often fail to scale and how architectural constraints, overhead, and memory behavior shape real performance.

This module is delivered through guided in-class experiments and structured lab activities, reinforcing an experimental mindset rather than a single graded programming assignment. Given the limited advanced mathematics prerequisites in the Webster University Computer Science curriculum, the module emphasizes conceptual understanding and empirical analysis rather than formal mathematical derivations.

1. Definitions and Core Concepts

What Is Performance Engineering?

Performance engineering is the systematic process of:

measure → analyze → explain → improve

Students learn that performance is not just about making code “faster,” but about understanding *why* a program behaves the way it does on real hardware

(see Performance engineering notes).

Key Performance Metrics

- Runtime (execution time)
- Speedup: improvement relative to serial execution
- Efficiency: utilization of computational resources
- Scalability: behavior as problem size or core count increases

Students are repeatedly reminded that a serial baseline is mandatory and sets the upper bound on achievable speedup.

Scaling Laws

- Amdahl's Law explains why speedup saturates due to serial fractions and overhead.
- Gustafson's Law explains why larger problems can still scale.

The module emphasizes how these laws manifest especially strongly in Python, due to runtime overheads and memory effects

(see attached slides)

2. Measuring Performance Correctly in Python

Students practice correct performance measurement techniques:

- Using `time.perf_counter()` for wall-clock timing
- Avoiding `time.time()` for benchmarking
- Running **multiple trials** and reporting median runtime
- Understanding the difference between **wall-clock time** and **CPU time**
- Recognizing and avoiding common mistakes:
 - Single-run timing
 - Timing I/O
 - Changing inputs between runs

Short live coding examples illustrate how misleading results can arise from improper timing.

3. Parallel Overhead in Python

This module makes overhead visible and unavoidable.

Sources of Overhead

- Process creation and teardown
- Task scheduling
- Inter-process communication
- Data serialization (pickling)
- Synchronization and waiting

Students compare fine-grained vs. coarse-grained parallelism, observing that naïve parallel programs can be slower than serial ones when overhead dominates.

In-class experiments demonstrate:

- Why chunk size matters
- Why pool reuse matters
- How data movement costs can dwarf computation costs

4. Compute-Bound vs. Memory-Bound Behavior

Students learn to distinguish:

- Compute-bound workloads (dominated by arithmetic)
- Memory-bound workloads (dominated by data movement)

Key ideas introduced:

- Arithmetic intensity
- Memory bandwidth saturation
- Why Python often becomes memory-bound early

Simple experiments contrast:

- `np.sum()` (memory-bound)
- Loops with repeated arithmetic (more compute-bound)

This distinction explains why adding cores often stops helping long before full CPU utilization.

5. Performance Visualization

Rather than relying on raw numbers, students learn to interpret plots:

- Runtime vs. cores
- Speedup vs. cores
- Efficiency vs. cores

Visualization reveals:

- Diminishing returns
- Saturation points
- Over-parallelization effects

A recurring lesson:

Numbers tell you what happened; plots tell you why.

6. NUMA Effects and Memory Locality

The module introduces NUMA (Non-Uniform Memory Access) at a conceptual and practical level:

- First-touch memory allocation
- Local vs. remote memory access

- Why performance degrades at socket boundaries

Students observe that:

- Memory placement can destroy scaling
- High CPU utilization does not guarantee good performance
- These effects occur **even in Python**

Live demonstrations contrast poor vs. improved memory locality strategies

See slides

7. Pedagogical Notes for Instructors

- There is no textbook
- I did not assign a homework:
 - No single homework emphasizes experimentation over implementation.
- Short, focused labs encourage hypothesis–test–explain cycles.
- Python is used deliberately to expose overhead and limits early, not to hide them.
- This module reinforces the idea that algorithmic improvements often beat parallelism.
- A key challenge in PUI settings is developing PDC homework that maintains conceptual rigor without relying on advanced mathematics, which necessitates a greater emphasis on empirical reasoning and experimental evidence.

8. The Use of HPC in Teaching Performance Engineering

While many performance-engineering concepts can be introduced on personal laptops, meaningful instruction benefits greatly from access to shared high-performance computing (HPC) systems. In this module, HPC resources were used to expose students to realistic performance behavior that is difficult to observe on small-scale machines, including core saturation, memory bandwidth limits, NUMA effects, and variability due to shared resource contention. Building on a separate instructional project, the author incorporated HPC-based experiments into performance-engineering instruction by guiding students through structured measurement, analysis, and explanation workflows on a production cluster. This approach enabled students to work with larger problem sizes, conduct controlled scaling studies, and experience authentic performance trade-offs encountered in real-world computational environments. The specific instructional steps used to integrate HPC into the classroom are outlined below.

Steps	Expected Learning Outcome
Step 0: Foundations: AI + HPC	Students gain essential background in AI workloads, data preprocessing, CPU execution pipelines, and the role of HPC in accelerating model training. They develop foundational performance-engineering concepts, including runtime, speedup, efficiency, and bottleneck identification.
Step 1: Baseline Model Introduction	Students learn the structure of compact deep-learning workloads, including dataset organization, model layers, forward and backward

	passes, and the end-to-end training loop. They execute a small baseline model locally to build intuition about correctness and runtime behavior.
Step 2: HPC and SLURM Orientation	Students gain operational fluency in accessing the HPC cluster, managing software environments, loading modules, organizing data directories, and writing reproducible SLURM job scripts. They learn to parameterize jobs using CPU core counts, memory requests, and parallel worker configurations.
Step 3: Controlled CPU Scaling Experiments	Students execute the same model using systematically varied CPU core counts (1, 2, 4, 8, 16, ...) and across heterogeneous node types. They practice controlled experimentation by fixing random seeds, batch sizes, data paths, and isolating a single variable per experiment.
Step 4: Performance Data Collection	Students collect clean and reproducible performance data, including training time per epoch, total runtime, speedup, efficiency, CPU utilization, memory usage, and data-loader throughput. They apply best practices such as warm-up runs, multiple trials, and median-of-three timing.
Step 5: Architectural and Scaling Analysis	Students interpret performance curves to identify scaling limits, detect memory-bound behavior, and reason about NUMA effects, I/O bottlenecks, and small-batch inefficiencies. They compare performance across different CPU architectures (e.g., EPYC vs. Xeon) and relate observations to hardware characteristics.
Step 6: Apply Optimization Techniques	Students apply targeted optimization techniques by modifying one parameter at a time and conducting short, replicated trials. They tune data-loader worker counts, adjust batch sizes, control thread placement and affinity (e.g., OMP_NUM_THREADS, taskset), enable or disable prefetching, and compare I/O layouts (shared vs. local storage). For each configuration, students rerun experiments, record changes in throughput, CPU and memory utilization, and time per epoch, and explain why performance improved or degraded in terms of architectural factors such as oversubscription, NUMA locality, caching effects, and I/O saturation.
Step 7: Synthesis, Interpretation, and Reporting	Students synthesize results into a coherent performance narrative, explaining why efficiency degrades at higher core counts, identifying dominant bottlenecks, and articulating interactions between software design and hardware architecture. Results are communicated using figures, tables, and evidence-based explanations grounded in performance-engineering principles.
Step 8: Model Extension and Deeper Exploration	Students extend the handwriting-recognition workload by increasing task complexity, such as adding multi-digit sequence recognition, expanding to additional handwritten alphabets, or experimenting with deeper convolutional neural networks. They evaluate how increased model complexity affects computational load, scaling behavior, and optimization effectiveness, reinforcing the relationship between algorithm design choices and HPC performance.

9. Included Materials

- Lecture slides: *Basic Performance Engineering with Python*
- Class notes: *Performance Engineering with Python*
- In-class timing and overhead experiments
- Visualization examples
- NUMA demonstrations

*Performance engineering is not about making code fast.
It is about understanding why code behaves the way it does.*